

AI Assisted Coding

ASSIGNMENT 5.1 & 6.1

Name: M.Goutham

HT No: 2303A52010

Batch: 31

Question 1: Employee Data

Task:

Create Python code that defines a class named

`Employee` with the following attributes: `empid`, `empname`, `designation`, `basic\_salary`, and `exp`. Implement a method `display\_details()` to print all employee details. Implement another method `calculate\_allowance()` to determine additional allowance based on experience:

- If `exp > 10 years` → allowance = 20% of `basic\_salary`
- If $5 \leq \text{exp} \leq 10$ years → allowance = 10% of `basic\_salary`
- If `exp < 5 years` → allowance = 5% of `basic\_salary`

Finally, create at least one instance of the `Employee` class, call the `display\_details()` method, and print the calculated allowance.

Prompt:

Create a Python class Employee with attributes empid, empname, designation, basic_salary, and exp. Add methods to display employee details and calculate allowance based on years of experience with 20% for >10 years, 10% for 5-10 years, and 5% for <5 years.

Code:

```
class Employee:

    def __init__(self, empid, empname, designation, basic_salary, exp):

        self.empid = empid

        self.empname = empname

        self.designation = designation

        self.basic_salary = basic_salary

        self.exp = exp


    def display_details(self):

        print(f"Employee ID: {self.empid}")

        print(f"Name: {self.empname}")

        print(f"Designation: {self.designation}")

        print(f"Basic Salary: {self.basic_salary}")

        print(f"Experience: {self.exp} years")


    def calculate_allowance(self):

        if self.exp > 10:

            allowance = self.basic_salary * 0.20

        elif self.exp >= 5:

            allowance = self.basic_salary * 0.10

        else:

            allowance = self.basic_salary * 0.05

        return allowance
```

```
def display_with_allowance(self):  
  
    self.display_details()  
  
    allowance = self.calculate_allowance()  
  
    total_salary = self.basic_salary + allowance  
  
    print(f"Allowance: {allowance}")  
  
    print(f"Total Salary: {total_salary}")  
  
  
# Example usage  
  
if __name__ == "__main__":  
  
    emp1 = Employee(101, "John Doe", "Senior Developer", 50000, 12)  
  
    emp1.display_with_allowance()
```

Output(O/P)

```
Employee ID: 101  
Name: John Doe  
Designation: Senior Developer  
Basic Salary: 50000  
Experience: 12 years  
Allowance: 10000.0  
Total Salary: 60000.0
```

Explanation:

This class demonstrates object-oriented programming principles by encapsulating employee data and behavior. The `calculate_allowance()` method uses conditional statements to determine allowance percentage based on experience tiers. The implementation follows clean code practices with clear method names and proper data encapsulation.

Question 2: Electricity Bill Calculation

Task :

Electricity Bill Calculation- Create Python code that defines a class named `ElectricityBill` with attributes: `customer\_id`, `name`, and `units\_consumed`. Implement a method `display\_details()` to print customer details, and a method `calculate\_bill()` where: - Units $\leq 100 \rightarrow ₹5$ per unit - 101 to 300 units $\rightarrow ₹7$ per unit - More than 300 units $\rightarrow ₹10$ per unit Create a bill object, display details, and print the total bill amount.

Prompt:

Create Python code that defines a class named `ElectricityBill` with attributes: `customer\_id`, `name`, and `units\_consumed`. Implement a method `display\_details()` to print

customer details, and a method `calculate\_bill()` where:

- Units $\leq 100 \rightarrow ₹5$ per unit
- 101 to 300 units $\rightarrow ₹7$ per unit
- More than 300 units $\rightarrow ₹10$ per unit

Create a bill object, display details, and print the total bill amount.

Code:

```
class ElectricityBill:

    def __init__(self, customer_id, name, units_consumed):

        self.customer_id = customer_id

        self.name = name

        self.units_consumed = units_consumed

    def display_details(self):

        print(f"Customer ID: {self.customer_id}")

        print(f"Name: {self.name}")

        print(f"Units Consumed: {self.units_consumed}")
```

```
def calculate_bill(self):  
  
    if self.units_consumed <= 100:  
  
        bill = self.units_consumed * 5  
  
    elif self.units_consumed <= 300:  
  
        bill = self.units_consumed * 7  
  
    else:  
  
        bill = self.units_consumed * 10  
  
    return bill  
  
  
# Create a bill object  
  
bill = ElectricityBill("C001", "John Doe", 250)  
  
  
# Display details  
  
bill.display_details()  
  
  
# Print total bill amount  
  
print(f"Total Bill Amount: {bill.calculate_bill()}")
```

Output(O/P)

```
Customer ID: C001  
Name: John Doe  
Units Consumed: 250  
Total Bill Amount: 1750
```

Explanation:

The `calculate_bill()` method implements tiered pricing logic commonly used in utility billing systems. This progressive pricing structure encourages energy conservation by charging higher rates for excessive consumption. The implementation correctly handles boundary conditions between pricing tiers.

Question 3: Product Discount Calculation

Task :

Create Python code that defines a class named `Product` with attributes: `product\_id`, `product\_name`, `price`, and `category`. Implement a method `display\_details()` to print product details. Implement another method `calculate\_discount()` where: - Electronics → 10% discount - Clothing → 15% discount - Grocery → 5% discount Create at least one product object, display details, and print the final price after discount.

Prompt:

Create a Python class Product with category-based discount calculation: 10% for Electronics, 15% for Clothing, and 5% for Grocery. Include methods to display product information and calculate discounted price

Code:

```
class Product:

    # Discount rates by category

    DISCOUNTS = {

        "Electronics": 0.10,

        "Clothing": 0.15,

        "Grocery": 0.05

    }

    def __init__(self, name, category, price):

        self.name = name

        self.category = category

        self.price = price

    def display_info(self):

        """Display product information"""
```



```
print(f"Product: {self.name}")

print(f"Category: {self.category}")

print(f"Price: ${self.price:.2f}")


def calculate_discounted_price(self):

    """Calculate and return the discounted price"""

    discount_rate = self.DISCOUNTS.get(self.category, 0)

    discount_amount = self.price * discount_rate

    discounted_price = self.price - discount_amount

    return discounted_price


def display_discount_info(self):

    """Display product info with discount details"""

    self.display_info()

    discount_rate = self.DISCOUNTS.get(self.category, 0)

    discounted_price = self.calculate_discounted_price()

    print(f"Discount: {discount_rate * 100:.0f}%")

    print(f"Discounted Price: ${discounted_price:.2f}")


# Example usage

if __name__ == "__main__":

    product1 = Product("Laptop", "Electronics", 1000)

    product1.display_discount_info()

    print()

    product2 = Product("T-Shirt", "Clothing", 50)
```

```
product2.display_discount_info()

print()

product3 = Product("Milk", "Grocery", 5)

product3.display_discount_info()
```

Output(O/P)

```
Product: Laptop
Category: Electronics
Price: $1000.00
Discount: 10%
Discounted Price: $900.00
```

```
Product: T-Shirt
Category: Clothing
Price: $50.00
Discount: 15%
Discounted Price: $42.50
```

```
Product: Milk
Category: Grocery
Price: $5.00
Discount: 5%
Discounted Price: $4.75
```

Explanation:

This class demonstrates polymorphic behavior through category-based discount calculation.

Question 4: Book Late Fee Calculation

Task :

Create Python code that defines a class named `LibraryBook` with attributes: `book\_id`, `title`, `author`, `borrower`, and `days\_late`. Implement a method `display\_details()` to print book details, and a method `calculate\_late\_fee()` where: - Days late $\leq 5 \rightarrow ₹5$ per day - 6 to 10 days late $\rightarrow ₹7$ per day - More than 10 days late $\rightarrow ₹10$ per day Create a book object, display details, and print the late fee.

Prompt:

Create a Python class LibraryBook to calculate late return fees with progressive rates: ₹5/day for ≤ 5 days, ₹7/day for 6-10 days, and ₹10/day for >10 days late. Include book details and display functionality.

Code:

```
class LibraryBook:

    def __init__(self, title, author, isbn, due_date):

        self.title = title

        self.author = author

        self.isbn = isbn

        self.due_date = due_date

    def calculate_late_fee(self, days_late):

        """Calculate late return fee based on progressive rates"""

        if days_late <= 0:

            return 0

        elif days_late <= 5:

            return days_late * 5

        elif days_late <= 10:

            return days_late * 7
```

```

        else:

            return days_late * 10

def display_details(self):

    """Display book details"""

    print(f"Title: {self.title}")

    print(f"Author: {self.author}")

    print(f"ISBN: {self.isbn}")

    print(f"Due Date: {self.due_date}")

def display_with_fee(self, days_late):

    """Display book details along with late fee"""

    self.display_details()

    fee = self.calculate_late_fee(days_late)

    print(f"Days Late: {days_late}")

    print(f"Late Fee: {fee}")

# Example usage

if __name__ == "__main__":

    book = LibraryBook("Python Programming", "Guido van Rossum", "978-0134957242",
"2025-02-01")

    book.display_with_fee(7)

```

Output(O/P)

Title: Python Programming
Author: Guido van Rossum
ISBN: 978-0134957242
Due Date: 2025-02-01
Days Late: 7
Late Fee: 49

Explanation:

The progressive penalty structure makes timely returns. The current approach is simpler but charges retroactively at the highest applicable rate.

Question 5: Student Performance Report

Task :

Define a function `student\_report(student\_data)` that accepts a dictionary containing student names and their marks. The function should: - Calculate the average score for each student - Determine pass/fail status (pass ≥ 40) - Return a summary report as a list of dictionaries Use Copilot suggestions as you build the function and format the output.

Prompt:

Create a Python function to generate student performance reports from a dictionary of names and marks. Calculate average scores, determine pass/fail status (passing threshold: 40), and return structured report data.

Code:

```
def generate_performance_report(students):  
  
    """  
  
    Generate student performance reports from a dictionary of names and marks.  
  
    Args:  
  
        students (dict): Dictionary with student names as keys and marks as values  
  
    Returns:  
  
        dict: Report containing individual results and summary statistics  
    """  
  
    if not students:  
  
        return {"students": [], "average": 0, "pass_count": 0, "fail_count": 0}  
  
    report = {  
  
        "students": [],
```

```
        "average": 0,

        "pass_count": 0,

        "fail_count": 0

    }

    total_marks = 0

    pass_threshold = 40

    for name, marks in students.items():

        status = "Pass" if marks >= pass_threshold else "Fail"

        if status == "Pass":

            report["pass_count"] += 1

        else:

            report["fail_count"] += 1

        report["students"].append({

            "name": name,

            "marks": marks,

            "status": status

        })

        total_marks += marks

    report["average"] = total_marks / len(students)

    return report
```

```
# Example usage

if __name__ == "__main__":

    students = {

        "Alice": 85,

        "Bob": 35,

        "Charlie": 92,

        "Diana": 45

    }

    report = generate_performance_report(students)

    print(report)
```

Output(O/P)

```
{'students': [{'name': 'Alice', 'marks': 85, 'status': 'Pass'}, {'name': 'Bob', 'marks': 35, 'status': 'Fail'}, {'name': 'Charlie', 'marks': 92, 'status': 'Pass'}, {'name': 'Diana', 'marks': 45, 'status': 'Pass'}], 'average': 64.25, 'pass_count': 3, 'fail_count': 1}
```

Explanation:

This function demonstrates functional programming concepts and data transformation. It processes heterogeneous input (lists or single values) robustly using `isinstance()` checks. The function returns structured data (list of dictionaries) rather than printing directly, promoting reusability and separation of concerns. The output formatting loop demonstrates clean data presentation.

Question 6: Taxi Fare Calculation

Task :

Create Python code that defines a class named `TaxiRide` with attributes: `ride\_id`, `driver\_name`, `distance\_km`, and `waiting\_time\_min`. Implement a method `display\_details()` to print ride details, and a method `calculate\_fare()` where: - ₹15 per km for the first 10 km - ₹12 per km for the next 20 km - ₹10 per km above 30 km - Waiting charge: ₹2 per minute Create a ride object, display details, and print the total fare.

Prompt:

Create Python code that defines a class named `TaxiRide` with attributes: `ride\_id`, `driver\_name`, `distance\_km`, and `waiting\_time\_min`. Implement a method `display\_details()` to print ride details, and a method `calculate\_fare()` where:

- ₹15 per km for the first 10 km
- ₹12 per km for the next 20 km
- ₹10 per km above 30 km
- Waiting charge: ₹2 per minute

Create a ride object, display details, and print the total fare.

Code:

```
class TaxiRide:

    def __init__(self, ride_id, driver_name, distance_km, waiting_time_min):

        self.ride_id = ride_id

        self.driver_name = driver_name

        self.distance_km = distance_km

        self.waiting_time_min = waiting_time_min
```

```
def display_details(self):

    print(f"Ride ID: {self.ride_id}")

    print(f"Driver Name: {self.driver_name}")

    print(f"Distance: {self.distance_km} km")

    print(f"Waiting Time: {self.waiting_time_min} minutes")


def calculate_fare(self):

    distance_fare = 0

    remaining_distance = self.distance_km

    # First 10 km at ₹15/km

    if remaining_distance <= 10:

        distance_fare += remaining_distance * 15

    else:

        distance_fare += 10 * 15

        remaining_distance -= 10

    # Next 20 km at ₹12/km

    if remaining_distance <= 20:

        distance_fare += remaining_distance * 12

    else:

        distance_fare += 20 * 12

        remaining_distance -= 20

    # Above 30 km at ₹10/km

    distance_fare += remaining_distance * 10

    # Waiting charge
```

```
        waiting_charge = self.waiting_time_min * 2

    # Total fare

    total_fare = distance_fare + waiting_charge

    return total_fare


# Create a ride object

ride = TaxiRide("R001", "Raj Kumar", 35, 10)


# Display details

ride.display_details()


# Print total fare

total_fare = ride.calculate_fare()

print(f"Total Fare: {total_fare}")
```

Output(O/P)

```
Ride ID: R001
Driver Name: Raj Kumar
Distance: 35 km
Waiting Time: 10 minutes
Total Fare: 460
```

Explanation:

This fare calculation demonstrates compound pricing with both distance and time components. waiting charges are included in the distance calculation.

The tiered structure reflects real-world taxi pricing where longer distances have reduced per-km rates.

Question 7: Statistics Subject Performance

Task :

Create a Python function

`statistics\_subject(scores\_list)` that accepts a list of 60 student scores and computes key performance statistics. The function should return the following:

- Highest score in the class
- Lowest score in the class
- Class average score
- Number of students passed (score ≥ 40)
- Number of students failed (score < 40)

Allow Copilot to assist with aggregations and logic

Prompt:

Create a Python function to analyze class performance from 60 student scores. Calculate highest, lowest, average scores, and count pass/fail students using a score threshold of 40.

Code:

```
def analyze_class_performance(scores):  
  
    """  
  
    Analyze class performance from student scores.  
  
    Args:  
  
        scores: List of 60 student scores  
  
    Returns:  
  
        Dictionary with highest, lowest, average scores and pass/fail counts
```

```
"""

if not scores or len(scores) == 0:

    return None


pass_threshold = 40


highest = max(scores)

lowest = min(scores)

average = sum(scores) / len(scores)


passed = sum(1 for score in scores if score >= pass_threshold)

failed = len(scores) - passed


return {

    "highest": highest,

    "lowest": lowest,

    "average": round(average, 2),

    "passed": passed,

    "failed": failed,

    "total_students": len(scores)

}


# Example usage

if __name__ == "__main__":

    student_scores = [45, 52, 38, 61, 55, 42, 39, 58, 67, 41,

                      48, 53, 36, 62, 51, 44, 59, 65, 40, 47,

                      54, 37, 63, 49, 46, 60, 35, 64, 50, 43,
```

```
56, 38, 66, 52, 45, 61, 39, 58, 48, 42,  
57, 51, 44, 68, 53, 41, 60, 47, 62, 36,  
55, 49, 43, 67, 54, 46, 59, 38, 64, 52]
```

```
result = analyze_class_performance(student_scores)  
  
print(result)
```

Output(O/P)

```
{'highest': 68, 'lowest': 35, 'average': 50.92, 'passed': 51, 'failed': 9, 'total_students': 60}
```

Explanation:

This function showcases effective use of Python built-in functions (max, min, sum) and list comprehensions for efficient data aggregation. The validation check ensures data integrity. The generator expression in sum() provides memory-efficient counting.

Lab 5: Ethical Foundations – Responsible AI Coding Practices

Question 8: Transparency in Algorithm Optimization

Task :

Use AI to generate two solutions for checking prime numbers: • Naive approach(basic) • Optimized approach Prompt: “Generate Python code for two prime-checking methods and explain how the optimized version improves performance.” Expected Output: • Code for both methods. • Transparent explanation of time complexity. • Comparison highlighting efficiency improvements.

Prompt:

Generate Python code for two prime-checking methods (naive and optimized) with transparent explanations of time complexity and performance improvements. Include clear comparisons of algorithmic efficiency

Code:

```
def is_prime_naive(n):  
  
    """Check if a number is prime using a naive approach.  
  
    Time Complexity:  $O(n)$   
  
    This method checks for factors from 2 to  $n-1$ .  
  
    It is inefficient for large numbers as it performs  $n-2$  checks.  
  
    """  
  
    if n <= 1:  
  
        return False  
  
    for i in range(2, n):  
  
        if n % i == 0:  
  
            return False  
  
    return True
```

```
def is_prime_optimized(n):  
  
    """Check if a number is prime using an optimized approach.  
  
    Time Complexity:  $O(\sqrt{n})$   
  
    This method checks for factors only up to the square root of  $n$ .  
  
    It significantly reduces the number of checks, especially for large numbers.  
  
    """  
  
    if n <= 1:  
  
        return False  
  
    if n <= 3:  
  
        return True  
  
    if n % 2 == 0 or n % 3 == 0:  
  
        return False
```



```

i = 5

while i * i <= n:

    if n % i == 0 or n % (i + 2) == 0:

        return False

    i += 6

return True

# Example usage

number = 29

print(f"Naive method: {number} is prime? {is_prime_naive(number)}")

print(f"Optimized method: {number} is prime? {is_prime_optimized(number)}")

```

Output(O/P)

```

Naive method: 29 is prime? True
Optimized method: 29 is prime? True

```

Explanation:

The naive prime check tests all numbers from 2 to n minus one to see if any divides the given number, which makes it slow with time complexity $O(n)$. The optimized method improves this by checking divisibility only up to the square root of the number, skipping even numbers and multiples of three, which greatly reduces the number of checks and runs in $O(\text{square root of } n)$ time, making it much faster for larger values.

Question 9: Transparency in Recursive Algorithms

Task :

Use AI to generate a recursive function to calculate Fibonacci numbers. Instructions: 1. Ask AI to add clear comments explaining recursion. 2. Ask AI to explain base cases and recursive calls. Expected Output: • Well-commented

recursive code. • Clear explanation of how recursion works. • Verification that explanation matches actual execution.

Prompt:

Generate a well-commented recursive Fibonacci function in Python. Explain base cases, recursive calls, and provide visualization of how recursion works with clear documentation.

Code:

```
def fibonacci(n):  
    """  
    Calculate the nth Fibonacci number using recursion.  
  
    Base cases:  
    - fibonacci(0) = 0  
    - fibonacci(1) = 1  
  
    Recursive case:  
    - fibonacci(n) = fibonacci(n-1) + fibonacci(n-2)  
  
    Args:  
    n: Non-negative integer representing the position in Fibonacci sequence  
  
    Returns:  
    The nth Fibonacci number  
  
    Example:  
    fibonacci(5) = 5  
    Sequence: 0, 1, 1, 2, 3, 5, 8, 13...  
    """  
    # Base cases: stop recursion
```

```

if n == 0:

    return 0

if n == 1:

    return 1


# Recursive case: break problem into smaller subproblems

return fibonacci(n - 1) + fibonacci(n - 2)


def fibonacci_with_trace(n, depth=0):

    """

    Fibonacci with recursion tree visualization.

    Shows how function calls branch during recursion.

    """

    indent = "  " * depth

    print(f"{indent} fibonacci({n})")


    if n == 0:

        print(f"{indent} returns 0 (base case)")

        return 0

    if n == 1:

        print(f"{indent} returns 1 (base case)")

        return 1


    # Recursive calls shown with indentation

    left = fibonacci_with_trace(n - 1, depth + 1)

    right = fibonacci_with_trace(n - 2, depth + 1)

    result = left + right

```

```
    print(f"{indent} returns {result}")

    return result


# Test the functions

if __name__ == "__main__":

    print("Fibonacci(5):", fibonacci(5))

    print("\nRecursion tree for fibonacci(4):")

    fibonacci_with_trace(4)
```

Output(O/P)

```
Fibonacci(5): 5
```

```
Recursion tree for fibonacci(4):
```

```
fibonacci(4)
  fibonacci(3)
    fibonacci(2)
      fibonacci(1)
        returns 1 (base case)
      fibonacci(0)
        returns 0 (base case)
    returns 1
  fibonacci(1)
    returns 1 (base case)
  returns 2
fibonacci(2)
  fibonacci(1)
    returns 1 (base case)
  fibonacci(0)
    returns 0 (base case)
  returns 1
returns 3
```

Explanation:

This program calculates Fibonacci numbers using recursion, where the function returns zero for input zero and one for input one, and for any larger value it calls itself to add the two previous Fibonacci numbers. The tracing version prints each function call with indentation to visually show how recursion branches into smaller subproblems, helping to understand the recursion tree and how results are combined step by step, though this approach is inefficient due to repeated calculations.

Question 10: Transparency in Error Handling

Task :

Use AI to generate a Python program that reads a file and processes data.
Prompt: "Generate code with proper error handling and clear explanations for

each exception.” Expected Output: • Code with meaningful exception handling. • Clear comments explaining each error scenario. • Validation that explanations align with runtime behavior.

Prompt:

Generate Python code with comprehensive error handling using custom exceptions. Include clear explanations for each error scenario: invalid input, calculation errors, and missing data. Demonstrate transparent exception handling with descriptive messages.

Code:

```
"""
Comprehensive error handling with custom exceptions.
Demonstrates transparent exception handling with descriptive messages.
"""

# Custom Exception Classes

class InvalidInputError(Exception):

    """Raised when input validation fails."""

    pass

class CalculationError(Exception):

    """Raised when a calculation operation fails."""

    pass

class MissingDataError(Exception):

    """Raised when required data is missing."""

    pass

# Example functions demonstrating error handling
```

```

def divide_numbers(a, b):

    """

    Divides two numbers with error handling.

    Args:

        a: Numerator

        b: Denominator

    Raises:

        InvalidInputError: If inputs are not numeric

        CalculationError: If division by zero is attempted

    """

    try:

        if not isinstance(a, (int, float)) or not isinstance(b, (int, float)):

            raise InvalidInputError(f"Expected numeric values, got {type(a).__name__}
and {type(b).__name__}")

        if b == 0:

            raise CalculationError("Cannot divide by zero")

        return a / b

    except (InvalidInputError, CalculationError) as e:

        print(f"Error: {e}")

        raise


def process_user_data(user_dict, required_fields):

    """

```

Validates and processes user data.

Args:

user_dict: Dictionary containing user information

required_fields: List of required field names

Raises:

MissingDataError: If required fields are missing

"""

```
missing_fields = [field for field in required_fields if field not in user_dict]
```

```
if missing_fields:
```

```
    raise MissingDataError(f"Missing required fields: {'',  
'.'.join(missing_fields)}")
```

```
return user_dict
```

```
# Main execution with comprehensive error handling
```

```
if __name__ == "__main__":
```

```
    # Test 1: Valid calculation
```

```
    try:
```

```
        result = divide_numbers(10, 2)
```

```
        print(f" Division result: {result}")
```

```
    except Exception as e:
```

```
        print(f" Unexpected error: {e}")
```

```
# Test 2: Invalid input (type error)
```

```
try:
```



```
    result = divide_numbers("10", 2)

except InvalidInputError as e:

    print(f" Caught invalid input: {e}")


# Test 3: Calculation error (division by zero)

try:

    result = divide_numbers(10, 0)

except CalculationError as e:

    print(f" Caught calculation error: {e}")


# Test 4: Valid user data

try:

    user = {"name": "Alice", "email": "alice@example.com"}

    process_user_data(user, ["name", "email"])

    print(" User data validated successfully")

except MissingDataError as e:

    print(f" Data validation failed: {e}")


# Test 5: Missing data

try:

    user = {"name": "Bob"}

    process_user_data(user, ["name", "email", "phone"])

except MissingDataError as e:

    print(f" Caught missing data: {e}")
```

Output(O/P)

```
Division result: 5.0
Error: Expected numeric values, got str and int
Caught invalid input: Expected numeric values, got str and int
Error: Cannot divide by zero
Caught calculation error: Cannot divide by zero
User data validated successfully
Caught missing data: Missing required fields: email, phone
```

Explanation:

This program demonstrates robust error handling in Python by defining custom exceptions for invalid input, calculation failures, and missing data, making errors clear and meaningful. The divide function validates input types and prevents division by zero, while the user data function ensures all required fields are present, and the main block shows how different exceptions are raised, caught, and handled cleanly to keep the program reliable and easy to debug.